We are formalizing the native `7xtgv` display architecture in repository `apws/260906-7xtgv`.

This task must produce a proper **7xtgv v0.1 draft specification first**, before significant implementation work.

The existing VT100/RunCPM terminal is a working client/test environment and has diverged internally from the original native dual-plane 7xtgv design. Preserve it. Do not restructure it merely to make it look architecturally pure.

Do not perform open-ended trial/error or optimisation loops. At architectural checkpoints, report findings to the human and wait for direction when a real choice exists.

1. Fundamental display model

Native 7xtgv is primarily a character/TUI-oriented video architecture, intentionally simple enough to have a credible FPGA/CPLD-style implementation.

Canonical unscaled cell size:

```
6 × 10 physical pixels
```

Grid dimensions are implementation/profile dependent.

Important profile:

```
80 × 24 cells
= 480 × 240 physical pixels
```

Each visible cell contains four bytes:

```
CHAR_A   8 bits
CHAR_B   8 bits
ATTR     8 bits
EXTA     8 bits
```

At 80×24:

```
1920 cells × 4 bytes = 7680 bytes
```

CHAR_A and CHAR_B are separate planes. Do not conceptually collapse them into one character word.

Beneath the cell compositor there may be a separate full-resolution:

```
HARD DESKTOP / BACKGROUND
```

This can be:

- a solid color
- a generated background
- a full-resolution RGB image
- an external framebuffer/source

Its storage is NOT part of the cell format.

The renderer receives the desktop/background pixel corresponding to the physical output pixel and overlays the cell graphics onto it.

2. ATTR mode selector

ATTR bits 7..6 are always:

```
MM
```

Modes:

```
00  TEXT
01  WIND
10  INST
11  SEMI
```

Modes 00–10 use the same broad paper/ink compositor and differ primarily in glyph interpretation.

SEMI has its own encoding.

3. TEXT/WIND/INST common ATTR format

For modes 00, 01 and 10:

```
ATTR = MM PP IIII
```

where:

```
MM    mode
PP    paper/base color index 0..3
IIII  ink/base color index 0..15
```

This deliberately preserves the lower six bits of the historical `GGPPIIII` 7xtgv layout while changing the former glyph-bank selector into the rendering mode.

Each mode may have its own base color tables:

```
PAPER[mode][4]
INK[mode][16]
```

Thus TEXT, WIND and INST can have related but independently themed palettes while sharing the identical hardware encoding.

RGB565 is sufficient for the reference implementation.

4. EXTA for TEXT/WIND/INST

For modes 00–10:

```
EXTA = PPPP IIII
```

where:

```
PPPP = paper brightness 0..15
IIII = ink brightness 0..15
```

Use distinct naming in code to avoid confusing ATTR.IIII color index with EXTA.IIII brightness.

Recommended names:

```
PaperLevel
InkLevel
```

Semantics:

```
0   minimum/off/black
15  nominal full intensity
```

Brightness is NOT alpha.

Brightness affects only the selected paper or ink color, not the underlying hard desktop.

The specification should NOT mandate linear RGB multiplication.

An implementation may use:

```
BrightPaper[mode][paperColor][level]
BrightInk[mode][inkColor][level]
```

lookup tables.

This is preferable for simple HDL and allows perceptually tuned brightness curves.

Expected uses:

- dim inactive windows
- focus changes
- window fade/darken
- blinking
- context switching effects
- instrumentation intensity
- visual hierarchy

EXTA is expected to change much less frequently than ordinary character content.

Future fade-to-desktop alpha is a separate facility and must not be conflated with brightness.

5. Common TEXT/WIND/INST compositor

Preserve the useful behavior of the original native 7xtgv compositor.

For each physical pixel inside a 6×10 cell:

```
A = bit from CHAR_A glyph
B = bit from CHAR_B glyph
```

Interpret A as the INK mask.

Interpret B as the PAPER CUTOUT mask with historical/inverted polarity:

```
A=1           -> INK
A=0, B=0      -> PAPER
A=0, B=1      -> HARD DESKTOP
```

Thus the priority is:

```
INK
  over
PAPER
  over/replaced by
DESKTOP through B cutouts
```

This gives several important primitives:

Normal opaque text:

```
CHAR_A = text glyph
CHAR_B = blank/all-zero glyph
```

Opaque empty paper:

```
CHAR_A = blank
CHAR_B = blank
```

Fully transparent cell:

```
CHAR_A = blank
CHAR_B = full/all-one glyph
```

Text directly over desktop:

```
CHAR_A = text glyph
CHAR_B = full/all-one glyph
```

WIND can use shaped CHAR_B cutouts for rounded corners, shadows and nonrectangular chrome while CHAR_A supplies visible frame pixels.

This rule should be explicitly unit-tested.

6. Glyph representation

TEXT, WIND and INST each expose a logical 256-glyph namespace.

The specification describes the visible 6×10 bitmap, not how hardware must store it.

Reference C representation may use:

```
uint8_t Glyph[256][10]
```

Canonical row packing:

```
bit 7 = leftmost visible pixel
bit 6 = next
...
bit 2 = rightmost visible pixel
```

Bits 1..0 are zero/reserved in the byte-oriented reference representation.

Correct stale comments in existing source that disagree with the actual renderer.

An FPGA may use ROM/BRAM.

A CPLD implementation may generate WIND/INST glyphs procedurally instead of storing all 256 bitmaps.

The visible behavior is normative; the storage implementation is not.

7. TEXT mode — MM=00

TEXT is the ordinary programming/editor/TUI character mode.

Goal:

- excellent readable 6×10 programming text
- syntax highlighting via ink colors
- useful UI/editor symbols
- no requirement for national alphabets
- ASCII/ANSI-oriented

Preserve direct ASCII correspondence:

```
0x20..0x7E = normal printable ASCII
```

Use the existing modified QL-like font as the starting design.

Define the rest of the 256-character space in aligned ranges:

```
0x00..0x1F  visible control/debug/basic UI symbols
0x20..0x7F  ASCII + DEL/debug glyph
0x80..0x9F  arrows, cursors, bullets, triangles,
             check marks, radio/checkbox-like symbols
0xA0..0xBF  programming/math/operator symbols
0xC0..0xDF  blocks, shading, bars, tiny graphical helpers
0xE0..0xEF  additional editor/TUI symbols
0xF0..0xFF  reserved/user/diagnostic extension
```

Do not invent national character coverage merely to consume codes.

Generate a TEXT atlas PNG showing all 256 slots with hexadecimal row/column labels.

Only existing ASCII glyphs need to be considered stable initially. New extended glyphs may remain provisional in v0.1.

8. WIND mode — MM=01

WIND supplies window chrome and polished cell-based TUI geometry.

Make glyph indexes intentionally structured rather than an arbitrary collection.

Recommended encoding of the 8-bit WIND glyph code:

```
TT FF SSSS
```

where:

```
TT = visual theme/style 0..3
FF = functional family 0..3
SSSS = member/topology/state 0..15
```

Thus each visual theme occupies a power-of-two-aligned 64-glyph block:

```
0x00..0x3F theme 0
0x40..0x7F theme 1
0x80..0xBF theme 2
0xC0..0xFF theme 3
```

Initial themes:

```
0 = crisp/single-line
1 = heavy/double-line
2 = soft/rounded
3 = raised/beveled/shadow-oriented
```

The same low-six-bit function should mean approximately the same thing across all four themes.

Suggested functional families:

```
FF=00  frame topology
FF=01  separators/title/tab geometry
FF=10  scrollbar/slider/navigation geometry
FF=11  bevel/shadow/fill/button/decorative pieces
```

For FF=00, use the low nibble particularly intelligently:

```
SSSS = NESW connectivity mask
```

One bit for each connection direction.

This naturally provides the 16 possible line topologies:

- blank
- individual end connections
- horizontal
- vertical
- corners
- T junctions
- cross
- etc.

This makes generated window layouts easy to reason about and potentially very cheap in HDL.

Important rendering properties:

- horizontal joins must meet cell edges exactly

- vertical joins must meet top/bottom exactly
- corners/junctions must be seam-free
- corresponding functions across themes must retain geometric compatibility where possible
- rounded and beveled corners may use CHAR_B cutouts to expose the hard desktop outside the visual window shape

Generate the initial WIND glyphs programmatically if convenient and emit:

```
wind_glyphs.h
wind_atlas.png
```

The atlas is a human review artifact. Do not treat the first artistic result as frozen.

9. INST mode — MM=10

INST supplies small instrumentation/control-panel graphics:

- LEDs/lamps
- bargraphs
- meters
- knobs
- sliders
- switches
- buttons
- seven-segment-like elements
- indicators
- small waveform/status primitives

Use another deliberately structured namespace:

```
CCCC VVVV
```

where:

```
CCCC = instrument class 0..15
VVVV = value/state/frame 0..15
```

This means every class occupies one aligned 16-glyph block.

Proposed v0.1 classes:

```
0x0x  general/blank/fill/markers
0x1x  LED/lamp/status indicator
0x2x  horizontal bargraph level 0..15
0x3x  vertical bargraph level 0..15
0x4x  analog meter/needle position 0..15
0x5x  rotary knob position 0..15
0x6x  horizontal slider position 0..15
0x7x  vertical slider position 0..15
0x8x  seven-segment/hex digit 0..F
0x9x  switch/button/toggle states
0xAx  pointer/direction/status symbols
0xBx  waveform/sparkline primitives
0xCx  scale/graticule/tick primitives
0xDx  connector/port/device symbols
0xEx  animation/progress/spinner frames
0xFx  reserved/user-defined
```

Do not spend excessive time perfecting the artwork.

Create a sensible first 6×10 implementation so the architectural idea can be exercised.

The class/state mapping is more important than the first glyph appearance.

Generate:

```
inst_glyphs.h
inst_atlas.png
```

Procedural generation is encouraged where natural:

- bargraphs from VVVV
- needle/knob positions from VVVV
- slider positions from VVVV
- seven-segment hex from VVVV

This is useful both for the software reference model and as guidance for later HDL that may not want a complete INST ROM.

10. SEMI mode — MM=11

SEMI is a 2×5 logical semigraphics cell mapped onto the physical 6×10 cell.

Each semipixel is exactly:

```
3 × 2 physical pixels
```

For an 80×24 cell screen:

```
160 × 120 SEMI pixels
```

No SEMI glyph ROM exists.

Each plane directly encodes a ten-bit bitmap.

SEMI ATTR layout:

```
ATTR = 11 CC AA BB
```

where:

```
CC = high two bits of 10-bit palette index
AA = high two bits of plane A mask
BB = high two bits of plane B mask
```

EXTA in SEMI mode is:

```
low 8 bits of palette index
```

Thus:

```
MaskA =
    ((ATTR >> 2) & 3) << 8
    | CHAR_A

MaskB =
    (ATTR & 3) << 8
    | CHAR_B

PaletteIndex =
    ((ATTR >> 4) & 3) << 8
    | EXTA
```

Each ranges 0..1023.

SEMI bit geometry:

```
bit0 bit1
bit2 bit3
bit4 bit5
bit6 bit7
bit8 bit9
```

For physical pixel coordinates inside the cell:

```
SemiX = (x >= 3)
SemiY = y >> 1
BitIndex = SemiY*2 + SemiX
```

No generic divide/modulo is needed in HDL.

11. SEMI palette RAM

Use a unified palette representation:

```
1024 palettes
× 4 colors
× RGB565
```

Memory:

```
1024 × 4 × 2 bytes
= 8192 bytes
```

Pixel palette address can conceptually be:

```
PaletteAddress =
    (PaletteIndex << 2) | PixelIndex
```

This gives a simple 4096×16-bit palette RAM.

Support two global/configurable SEMI interpretations.

SEMI_OVERLAY

Preserves native two-plane overlay philosophy.

For each semipixel:

```
AB=00 -> hard desktop
AB=01 -> B color
AB=10 -> A color
AB=11 -> A color
```

A has priority over B.

The palette can still physically contain four entries; entry 0 is ignored in favor of desktop, and entries 2/3 may contain the same A color.

SEMI_INDEXED4

Full graphics/demo/game interpretation:

```
AB=00 -> palette color 0
AB=01 -> palette color 1
AB=10 -> palette color 2
AB=11 -> palette color 3
```

No transparency is implied.

Do not consume a per-cell bit to select these interpretations. Make the selection a global/display/surface configuration register.

This lets the same cell RAM architecture serve:

- overlays over a high-resolution desktop
- full-screen 160×120 four-color-local graphics

12. Hard desktop and future window compositor

The native pixel pipeline should conceptually be:

```
hard desktop pixel
      ↓
B/paper cutout decision
```

```
      ↓
A/ink overlay
      ↓
color/brightness lookup
      ↓
optional future alpha/fade stage
      ↓
output RGB
```

For SEMI_INDEXED4, the SEMI palette output replaces the normal desktop-overlay path.

Do not build a hardware window manager.

Future smart-terminal software owns:

- window buffers
- x/y/width/height
- clipping
- z-order
- focus
- titles
- app association
- overlap handling

It flattens currently visible window cells into native 7xtgv cell RAM.

Inactive-window dimming can be implemented by transforming EXTA brightness during flattening.

If a source cell already has a brightness value and a whole-window brightness factor is applied, software may combine them approximately as:

```
Effective =
    round(CellLevel * WindowLevel / 15)
```

No additional native-cell field is required.

Future fade-to-desktop/alpha is optional and separate.

A simple implementation could choose between cell output and desktop using spatial/temporal dithering rather than implement an RGB multiplier.

Do not implement this in v0.1.

13. HDL/resource perspective

Keep the spec behavior-oriented.

Storage and logic are separate concerns.

The logic should remain simple even if a small CPLD requires external RAM while an FPGA keeps the same resources in BRAM.

SEMI requires:

- two 10-bit mask registers
- simple x/y counters
- two bit selections
- one 10-bit palette index
- one small priority/index mux
- 4096×16 palette lookup for full Indexed4 operation

There is no 1024-glyph SEMI ROM.

WIND and INST may be:

- bitmap ROMs
- procedural generators
- hybrid ROM/procedural implementations

TEXT is naturally ROM-oriented.

Do not make the external behavioral specification depend on which implementation is selected.

14. Reference image converter

Create a desktop-side Python converter.

Input:

```
arbitrary PNG/JPEG/etc.
```

Pipeline begins:

```
decode RGB
crop/contain as requested
high-quality resize to target SEMI resolution
```

For the 80×24 demo profile:

```
160×120 target pixels
```

Important: resampling happens before target color quantization because high-quality resampling generates new intermediate colors even if the source was indexed/paletted.

Provide at least two conversion strategies.

Strategy A: SAFE14 / reference mode

This must be implemented first.

1. Resize image to 160×120 RGB.
2. Find an adaptive GLOBAL palette of at most 14 colors for that image.
3. Quantize the resized image to those 14 colors.
4. Process each 2×5 cell.
5. Select the best at-most-four colors from those 14 for that cell.
6. Encode its ten 2-bit pixel selections into MaskA/MaskB.
7. Map the selected four-color set to a SEMI palette index.
8. Emit only palette combinations actually used.

Mathematical reason for 14:

```
C(14,4) = 1001
```

There are only 1001 possible four-color subsets of a 14-color global palette.

Therefore every possible local four-color palette can fit into the 1024-entry SEMI palette namespace.

This eliminates the global palette-of-palettes/codebook-clustering problem entirely.

If a cell uses fewer than four colors, unused palette slots may contain arbitrary other members of the 14-color global palette; those indexes simply will not be referenced.

If a cell contains at most four distinct quantized colors, it can be reproduced exactly.

If it contains five or more, only the local four-color reduction contributes additional error.

This should be the deterministic reference/demo converter.

Do not populate unused SEMI palette entries unnecessarily.

Report:

- global colors actually used
- distinct four-color SEMI palettes used
- cells requiring reduction from >4 source colors
- reconstruction error

Strategy B: ADAPTIVE1024 / advanced mode

Implement only after SAFE14 works and the human reviews the result.

Operate from the full resized RGB image.

For each cell:

- derive an ideal local palette of up to four colors
- deduplicate local palettes
- if >1024 remain, merge/cluster them into at most 1024 palette entries
- reassign every cell to its lowest-error palette
- encode masks

Use a perceptually sensible color space if convenient.

Keep the algorithm deterministic.

Do not run many random restarts or open-ended quality searches.

SAFE14 remains the architectural reference even if ADAPTIVE1024 later looks better.

Optional explicit prequantization levels such as:

```
16
32
64
128
```

may be provided for experiments, but do not automatically benchmark all of them.

15. Converter output

Emit:

- reconstructed 160×120 preview PNG
- nearest-neighbor 480×240 preview PNG
- global palette report if applicable
- SEMI palette table
- CHAR_A
- CHAR_B
- ATTR
- EXTA
- compact asset/header suitable for embedded demo

Include basic asset metadata/versioning.

Suggested fields:

```
magic
version
columns
rows
mode
compositor
palette count
palette format
```

First implementation may simply store:

```
cell RAM      7680 bytes
full palette  8192 bytes
```

for a maximum raw payload around:

```
15872 bytes + tiny header
```

Optimise packing only later if useful.

16. Parrot/demo image

Use a user-provided or repository-local parrot/test image.

Do not waste effort searching for an arbitrary replacement if the intended source is absent; ask the human for the image.

First convert it with SAFE14.

Generate its global 14-color palette from the image itself.

Generate its SEMI palettes from the combinations actually required by the image.

Render it with SEMI_INDEXED4.

Show/report:

- source dimensions/color type
- resized target
- 14-color global palette
- number of SEMI palettes actually used
- number of cells with >4 quantized source colors
- palette bytes
- cell bytes
- total asset bytes
- reconstruction error
- preview image

Do not optimise based only on a numerical metric. Human visual inspection is a formal checkpoint.

17. Glyph generation/review tools

Create simple host-side tools that can generate and preview TEXT/WIND/INST glyph tables.

Output one clearly labeled 16×16 glyph atlas for each mode.

The atlas should make hexadecimal index structure obvious so the human can visually inspect:

- ASCII placement
- WIND theme equivalence
- WIND topology joins
- INST class/state progression

This human-visible atlas is an important design tool, not merely documentation.

Do not freeze provisional WIND/INST artwork without review.

18. Formal specification deliverable

Create a standalone specification document in the repository, for example:

```
docs/7xtgv-native-v0.1.md
```

It should contain:

- terminology
- cell memory layout
- bit diagrams
- mode definitions
- glyph row encoding
- compositing truth tables
- palette structures
- brightness semantics
- hard-desktop semantics
- SEMI geometry
- SEMI_OVERLAY
- SEMI_INDEXED4
- memory/resource tables
- normative behavior versus implementation suggestions
- compatibility notes for the existing demos/VT terminal
- reserved values/future-extension notes

Use normative wording where appropriate:

```
MUST
SHOULD
MAY
```

Clearly mark undecided/provisional areas.

19. Implementation order and human checkpoints

Proceed in this order:

A. specification only

Write the v0.1 native spec and proposed TEXT/WIND/INST index maps.

Show it to the human.

Do not proceed into broad implementation until architectural questions discovered here are resolved.

B. reference renderer

Implement platform-independent native cell decoding/compositing and tests.

C. glyph generators

Generate first TEXT/WIND/INST atlases.

Show PNG atlases to human.

D. SAFE14 converter

Generate reconstructed parrot preview and statistics.

Show human.

E. minimal on-device SEMI demo

Display the static asset with the least invasive integration possible.

Do not modify working VT100/RunCPM behavior.

F. only after review

Consider:

- ADAPTIVE1024 image conversion
- window compositor
- alpha/fade-to-desktop
- additional WIND/INST artwork
- procedural HDL-oriented glyph generators
- performance optimisation

At every checkpoint report assumptions, memory use and any newly discovered architectural choice.

Do not silently choose a complicated design if asking the human would be cheaper and safer.